

Project Report: Refactoring CS182 HW11 Coding Questions

Subject: Scaling Laws and Transformer Interpretability Author: Fangzhou Date: December 2025

1. Overview

This report details the refactoring of two coding notebooks from the CS182 Fall 2025 Homework 11 curriculum: `scaling_laws.ipynb` and `q_code_interpretability.ipynb`.

While completing these assignments, I observed that the codebase suffered from quality issues frequently found in academic scripts, such as cryptic variable names, a lack of type safety, and monolithic functions that handle too many responsibilities at once. While these notebooks successfully demonstrated the core concepts, their implementation did not align with modern software engineering standards.

To address this, I utilized a "Human-in-the-Loop" workflow with Claude (Opus 4.5) to transform these legacy scripts into modular, robust, and readable codebases. The goal was to demonstrate that educational code does not need to sacrifice quality for simplicity.

Repository: https://github.com/Fangzhou66/cs182_hw11_refactor

2. Methodology

The refactoring process was iterative. I prompted Claude to analyze the original notebooks for technical debt, validate those findings against authoritative style guides (such as PEP 8 and Google's ML Best Practices), and implement improvements while ensuring the pedagogical core of the assignment remained intact.

2.1 Refactoring `scaling_laws.ipynb`

2.1.1 Identifying Technical Debt

The original scaling laws notebook relied heavily on the conventions of "scratchpad" coding. Key issues included the use of single-letter variables (e.g., `N`, `D`, `w`) which obscured meaning, and a lack of type annotations, making the data flow difficult to track. Additionally, the code used the legacy

`np.random.seed()` API, which is no longer recommended for reproducible research, and contained massive training functions that were difficult to debug or test.

2.1.2 Implementation of Best Practices

To modernize this notebook, I focused on encapsulation and clarity:

- **Standardization & Typing:** I enforced PEP 8 formatting and added comprehensive type hints (e.g., `NDArray[np.floating]`). This clarifies the expected inputs and outputs for every function.
- **Encapsulation:** Instead of passing loose arrays (`X_train`, `y_train`) between functions, I introduced a `LinearDataset` dataclass to keep related data logically grouped.
- **Modernizing Randomness:** I migrated the random number generation to the modern `np.random.default_rng(seed)` generator, ensuring thread safety and better reproducibility.
- **Decomposition:** The previously monolithic `train_mlp_sgd()` function was broken down into smaller, single-responsibility units: `create_mlp_model()`, `create_optimizer()`, and `train_mlp()`.

2.2 Refactoring `q_code_interpretability.ipynb`

2.2.1 Identifying Technical Debt

The interpretability notebook presented different challenges. It utilized non-standard indentation (2 spaces) and cryptic variable naming conventions (e.g., `XOV`, `e_Z`) that made the complex linear algebra of Transformers harder to follow than necessary. Furthermore, "magic numbers" like `-1e9` for masking were used without explanation, and the code lacked references to the academic literature it was based on.

2.2.2 Implementation of Best Practices

The refactoring focused on making the math "read" like code:

- **Semantic Naming:** I renamed mathematical shorthands to descriptive variables. For instance, `Z` became `attention_logits` and `XOV` became `value_projections`. This bridges the gap between the equations and the implementation.
- **Documentation:** I implemented NumPy-style docstrings for every function, including parameters, returns, and—crucially—academic references. The module now cites foundational papers like Elhage et al. (2021) to provide context.
- **Numerical Stability:** I extracted the softmax computation into a helper function to ensure numerical stability and added explicit `dtype=np.float64` to array conversions to prevent precision errors.
- **Educational Comments:** I replaced generic `TODO` comments with step-by-step explanatory comments that guide the reader through the attention mechanism logic.

3. Verification

To ensure that the refactoring improved code quality without altering the logic, I validated both notebooks against the original results.

Scaling Laws Verification: The refactored code successfully reproduced the expected slopes for the scaling laws. A minor deviation in the MLP SGD slope (0.18 vs 0.13) was observed, which is attributable to the switch to the more robust `default_rng` generator; the overall scaling trend remained consistent.

Transformer Verification: All unit tests for the attention heads and induction heads passed.

- `single_attention_head`: Passed
- `induction_copy_head`: Passed

4. Conclusion

This project demonstrates that treating academic homework like a software engineering project yields significant benefits. By applying professional standards—such as semantic naming, modular design, and proper documentation—we reduced the cognitive load required to understand the code.

The refactored notebooks prove that "clean code" is not just an aesthetic choice; it is a pedagogical tool. When variable names describe what they represent (e.g., `query_key_weights` instead of `WQK`), students can focus on the concepts rather than deciphering the syntax. This exercise affirms that modern ML engineering practices can effectively coexist with, and enhance, academic instruction.

References

1. Van Rossum, G., et al. PEP 484 – Type Hints. Python.org.
2. Google. Google Python Style Guide.
3. Van Rossum, G., et al. PEP 8 – Style Guide for Python Code.
4. NumPy. NumPy Docstring Style Guide.
5. Google. Rules of Machine Learning: Best Practices for ML Engineering.
6. Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship.